Question 1

(a) Assumptions

i. Each form is one order. The form has no order number and no field may be added, so an order is identified by (customerNumber, orderDate, orderTime): a customer does not place two orders at the same date and time.

ii. Each order is handled by exactly one employee. Employees are not tied to specific customers.

iii. Employee names are unique, since the form has no employee ID.

iv. customerNumber uniquely identifies a customer, and determines the customer's name and type.

v. partNumber uniquely identifies a part, and determines its name, type, cage code and unit price.

vi. A cage stores only one type of part, so cageCode determines partType.

vii. Attributes are renamed to follow the class naming convention: Date → orderDate, Time → orderTime, Employee → employeeName, Name → partName, Type → partType.

viii. unitPrice is the current price of the part and does not change from order to order.

(b) Normalization process

Starting table (not in a normal form)

All of the data on the form is shown in one table. The part columns contain several values for a single order (4 parts), so the table has a repeating group and is not even in 1NF.

customerNumber	orderDate	orderTime	customerName	customerType	employeeName	partNumber	partName	partType	cageCode	quantityOrdered	unitPrice
HG54587	7/1/2024	10:30am	Jeff Peterson	Consumer	D. Harrison	10654	Float Control	Plumbing	G413	4	12
						10456	Modulator	Electrical	H433	3	7
						10776	Hose Assembly	Plumbing	G413	7	9
						10657	Float Assembly	Plumbing	G413	5	10

1NF

Reason: a repeating group is not allowed in 1NF, so each part ordered is placed on its own row and the order and customer information is repeated on every row. One order now spans several rows, so the PK becomes the combination (customerNumber, orderDate, orderTime, partNumber).

order_form (1NF)

customerNumber	**orderDate**	**orderTime**	**partNumber**	customerName	customerType	employeeName	partName	partType	cageCode	quantityOrdered	unitPrice
HG54587	7/1/2024	10:30am	10654	Jeff Peterson	Consumer	D. Harrison	Float Control	Plumbing	G413	4	12
HG54587	7/1/2024	10:30am	10456	Jeff Peterson	Consumer	D. Harrison	Modulator	Electrical	H433	3	7
HG54587	7/1/2024	10:30am	10776	Jeff Peterson	Consumer	D. Harrison	Hose Assembly	Plumbing	G413	7	9
HG54587	7/1/2024	10:30am	10657	Jeff Peterson	Consumer	D. Harrison	Float Assembly	Plumbing	G413	5	10

2NF

Reason: the PK is composite, so every non-key attribute must depend on the whole PK. Checking each one:

- customerNumber → customerName, customerType: depends on only part of the PK.
- (customerNumber, orderDate, orderTime) → employeeName: depends on only part of the PK (not on partNumber).
- partNumber → partName, partType, cageCode, unitPrice: depends on only part of the PK.
- (customerNumber, orderDate, orderTime, partNumber) → quantityOrdered: depends on the whole PK, so it stays.

Each partial dependency is moved into its own relation, with the attribute(s) it depends on as that relation's PK:

customer (2NF)

customerNumber	customerName	customerType
HG54587	Jeff Peterson	Consumer

part_order (2NF)

customerNumber	**orderDate**	**orderTime**	employeeName
HG54587	7/1/2024	10:30am	D. Harrison

part (2NF)

partNumber	partName	partType	cageCode	unitPrice
10654	Float Control	Plumbing	G413	12
10456	Modulator	Electrical	H433	7
10776	Hose Assembly	Plumbing	G413	9
10657	Float Assembly	Plumbing	G413	10

order_item (2NF)

customerNumber	**orderDate**	**orderTime**	**partNumber**	quantityOrdered
HG54587	7/1/2024	10:30am	10654	4
HG54587	7/1/2024	10:30am	10456	3
HG54587	7/1/2024	10:30am	10776	7
HG54587	7/1/2024	10:30am	10657	5

3NF

Reason: 3NF does not allow a non-key attribute to depend on another non-key attribute. In part, partNumber → cageCode and cageCode → partType (assumption 6), so partType depends on the PK only through cageCode, which is a transitive dependency. cageCode and partType are moved into a new relation cage, and cageCode stays in part as a foreign key. customer, part_order and order_item have no transitive dependencies (employeeName is a plain attribute of part_order), so they do not change; their FKs are now italicized.

customer (3NF)

customerNumber	customerName	customerType
HG54587	Jeff Peterson	Consumer

cage (3NF)

cageCode	partType
G413	Plumbing
H433	Electrical

part (3NF)

partNumber	partName	*cageCode*	unitPrice
10654	Float Control	G413	12
10456	Modulator	H433	7
10776	Hose Assembly	G413	9
10657	Float Assembly	G413	10

part_order (3NF)

customerNumber	**orderDate**	**orderTime**	employeeName
HG54587	7/1/2024	10:30am	D. Harrison

order_item (3NF)

customerNumber	***orderDate***	***orderTime***	***partNumber***	quantityOrdered
HG54587	7/1/2024	10:30am	10654	4
HG54587	7/1/2024	10:30am	10456	3
HG54587	7/1/2024	10:30am	10776	7
HG54587	7/1/2024	10:30am	10657	5

(c) 3NF relations (PKs in bold and underlined, FKs in italics)

customer(**customerNumber**, customerName, customerType)

cage(**cageCode**, partType)

part(**partNumber**, partName, *cageCode*, unitPrice)

FK: cageCode → cage

part_order(***customerNumber***, **orderDate**, **orderTime**, employeeName)

FK: customerNumber → customer

order_item(***customerNumber***, ***orderDate***, ***orderTime***, ***partNumber***, quantityOrdered)

FK: (customerNumber, orderDate, orderTime) → part_order

FK: partNumber → part

Question 2

(a) Assumptions

i. Each row is one appointment. A patient cannot have two appointments at the same date and time, so an appointment is identified by (patNo, appointmentDate, appointmentTime).

ii. Each appointment is with exactly one therapist at exactly one branch.

iii. staffNo uniquely identifies a therapist and determines therapistName.

iv. patNo uniquely identifies a patient and determines patName.

v. A therapist sees patients at only one branch on any given day, but may work at different branches on different days, so (staffNo, appointmentDate) determines branchNo.

vi. The grouped heading "appointment" is split into two attributes, renamed date → appointmentDate and time → appointmentTime. The other attribute names already follow the class naming convention.

vii. branchNo is the only branch attribute shown, so no separate branch relation is created.

(b) Normalization process

Starting table

All of the data is shown in one table. Every cell holds a single value and there is no repeating group.

therapist_appointment (starting table)

staffNo	therapistName	patNo	patName	appointmentDate	appointmentTime	branchNo
S1011	Fred Smith	P100	Lily White	9/12/2022	10:00	M15
S1011	Fred Smith	P105	Jill Baker	9/12/2022	12:00	M15
S1024	Heidi Pierce	P108	Andy McKee	9/12/2022	10:00	Q10
S1024	Heidi Pierce	P108	Andy McKee	9/14/2022	14:00	Q10
S1032	Richard Levin	P105	Jill Baker	9/14/2022	16:30	M15
S1032	Richard Levin	P110	Jimmy Winter	9/15/2022	18:00	B13

1NF

Reason: the table is already in 1NF because every cell holds a single value and there is no repeating group, so no data changes. The only step is to identify the PK. A patient has at most one appointment at a given date and time, so the PK is (patNo, appointmentDate, appointmentTime).

therapist_appointment (1NF)

staffNo	therapistName	**patNo**	patName	**appointmentDate**	**appointmentTime**	branchNo
S1011	Fred Smith	P100	Lily White	9/12/2022	10:00	M15
S1011	Fred Smith	P105	Jill Baker	9/12/2022	12:00	M15
S1024	Heidi Pierce	P108	Andy McKee	9/12/2022	10:00	Q10
S1024	Heidi Pierce	P108	Andy McKee	9/14/2022	14:00	Q10
S1032	Richard Levin	P105	Jill Baker	9/14/2022	16:30	M15
S1032	Richard Levin	P110	Jimmy Winter	9/15/2022	18:00	B13

2NF

Reason: the PK is composite, so every non-key attribute must depend on the whole PK. Checking each one:

- patNo → patName: depends on only part of the PK.
- (patNo, appointmentDate, appointmentTime) → staffNo: depends on the whole PK, because each appointment is with one therapist.
- therapistName and branchNo depend on the whole PK only indirectly, through staffNo. They are not partial dependencies, so they are handled in 3NF.

The partial dependency is moved into its own relation, with patNo as its PK:

patient (2NF)

patNo	patName
P100	Lily White
P105	Jill Baker
P108	Andy McKee
P110	Jimmy Winter

appointment (2NF)

patNo	**appointmentDate**	**appointmentTime**	staffNo	therapistName	branchNo
P100	9/12/2022	10:00	S1011	Fred Smith	M15
P105	9/12/2022	12:00	S1011	Fred Smith	M15
P108	9/12/2022	10:00	S1024	Heidi Pierce	Q10
P108	9/14/2022	14:00	S1024	Heidi Pierce	Q10
P105	9/14/2022	16:30	S1032	Richard Levin	M15
P110	9/15/2022	18:00	S1032	Richard Levin	B13

3NF

Reason: 3NF does not allow a non-key attribute to depend on anything other than the whole key. In appointment:

- staffNo → therapistName: one non-key attribute determines another (transitive dependency), so therapistName moves to a new relation therapist.
- (staffNo, appointmentDate) → branchNo: the determinant is a non-key attribute plus part of the PK, not a key of appointment (transitive dependency), so branchNo moves to a new relation therapist_branch with PK (staffNo, appointmentDate).
- appointment keeps staffNo. Its (staffNo, appointmentDate) is a composite FK to therapist_branch, which guarantees that the therapist has a branch on the day of the appointment.

patient (3NF)

patNo	patName
P100	Lily White
P105	Jill Baker
P108	Andy McKee
P110	Jimmy Winter

therapist (3NF)

staffNo	therapistName
S1011	Fred Smith
S1024	Heidi Pierce
S1032	Richard Levin

therapist_branch (3NF)

staffNo	**appointmentDate**	branchNo
S1011	9/12/2022	M15
S1024	9/12/2022	Q10
S1024	9/14/2022	Q10
S1032	9/14/2022	M15
S1032	9/15/2022	B13

appointment (3NF)

patNo	***appointmentDate***	**appointmentTime**	*staffNo*
P100	9/12/2022	10:00	S1011
P105	9/12/2022	12:00	S1011
P108	9/12/2022	10:00	S1024
P108	9/14/2022	14:00	S1024
P105	9/14/2022	16:30	S1032
P110	9/15/2022	18:00	S1032

(c) 3NF relations (PKs in bold and underlined, FKs in italics)

patient(**patNo**, patName)

therapist(**staffNo**, therapistName)

therapist_branch(***staffNo***, **appointmentDate**, branchNo)

FK: staffNo → therapist

appointment(***patNo***, ***appointmentDate***, **appointmentTime**, *staffNo*)

FK: patNo → patient

FK: (staffNo, appointmentDate) → therapist_branch

Question 3

(a) Assumptions

i. Each row records the hours one employee worked on one contract. An employee appears at most once per contract, so a row is identified by (eNo, contractNo).

ii. eNo uniquely identifies an employee (given) and determines eName.

iii. contractNo uniquely identifies a contract. Each contract applies to only one event (given), so contractNo → eventNo. An event can have several contracts.

iv. Each event takes place at one location, so eventNo → eventLoc.

v. hours is the total number of hours the employee worked under that contract.

vi. The attribute names already follow the class naming convention, so none are renamed.

(b) Normalization process

Starting table

All of the data is shown in one table. Every cell holds a single value and there is no repeating group.

employee_hours (starting table)

eNo	contractNo	hours	eName	eventNo	eventLoc
1135	C1024	16	Smith J	H25	Queens
1057	C1024	24	Hocine D	H25	Queens
1068	C1025	28	White T	H4	Yonkers
1135	C1025	15	Smith J	H4	Yonkers
1135	C1026	10	Smith J	H25	Queens

1NF

Reason: the table is already in 1NF because every cell holds a single value and there is no repeating group, so no data changes. The only step is to identify the PK. An employee can work on several contracts and a contract has several employees, so the PK is (eNo, contractNo).

employee_hours (1NF)

eNo	**contractNo**	hours	eName	eventNo	eventLoc
1135	C1024	16	Smith J	H25	Queens
1057	C1024	24	Hocine D	H25	Queens
1068	C1025	28	White T	H4	Yonkers
1135	C1025	15	Smith J	H4	Yonkers
1135	C1026	10	Smith J	H25	Queens

2NF

Reason: the PK is composite, so every non-key attribute must depend on the whole PK. Checking each one:

- eNo → eName: depends on only part of the PK.
- contractNo → eventNo, eventLoc: depends on only part of the PK.
- (eNo, contractNo) → hours: depends on the whole PK, so it stays.

Each partial dependency is moved into its own relation, with the attribute it depends on as that relation's PK:

employee (2NF)

eNo	eName
1135	Smith J
1057	Hocine D
1068	White T

contract (2NF)

contractNo	eventNo	eventLoc
C1024	H25	Queens
C1025	H4	Yonkers
C1026	H25	Queens

contract_employee (2NF)

eNo	**contractNo**	hours
1135	C1024	16
1057	C1024	24
1068	C1025	28
1135	C1025	15
1135	C1026	10

3NF

Reason: 3NF does not allow a non-key attribute to depend on another non-key attribute. In contract, contractNo → eventNo and eventNo → eventLoc, so eventLoc depends on the PK only through eventNo, which is a transitive dependency. eventNo and eventLoc are moved into a new relation event, and eventNo stays in contract as a foreign key. employee and contract_employee have no transitive dependencies, so they do not change; the FKs in contract_employee are now italicized.

employee (3NF)

eNo	eName
1135	Smith J
1057	Hocine D
1068	White T

event (3NF)

eventNo	eventLoc
H25	Queens
H4	Yonkers

contract (3NF)

contractNo	*eventNo*
C1024	H25
C1025	H4
C1026	H25

contract_employee (3NF)

eNo	***contractNo***	hours
1135	C1024	16
1057	C1024	24
1068	C1025	28
1135	C1025	15
1135	C1026	10

(c) 3NF relations (PKs in bold and underlined, FKs in italics)

employee(**eNo**, eName)

event(**eventNo**, eventLoc)

contract(**contractNo**, *eventNo*)

FK: eventNo → event

contract_employee(***eNo***, ***contractNo***, hours)

FK: eNo → employee

FK: contractNo → contract